Recursion

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **identify** the base case and the recursive case of a recursive method.
2. Be able to **write** a recursive method that makes progress toward its base case.
3. Be able to **explain** why a recursive method can throw a `StackOverflowError`.

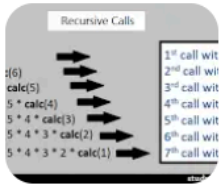
Google recursion

AI Mode All Shopping Images Videos News Forums More Tools

Did you mean: ***recursion***

AI Overview

Recursion is a process or function that calls itself within its own definition to solve a smaller, self-similar version of a problem. Wikipedia +1



Core Components

Every recursive solution needs two main parts:

- **Base Case:** The stopping condition that ends the recursion. Without it, the function runs forever and causes a stack overflow error.
- **Recursive Case:** The step where the function calls itself with a modified or smaller input, moving closer to the base case. GeeksforGeeks +1

Types of Recursion

- **Direct Recursion:** A function calls itself directly inside its own body.

Wikipedia
Recursion - Wikipedia
Recursion occurs when the definition of a concept or process depends on a simpler o...



Wikipedia
Recursion (computer science) - Wikipedia
Definition: A method of solving computational problems where a function...

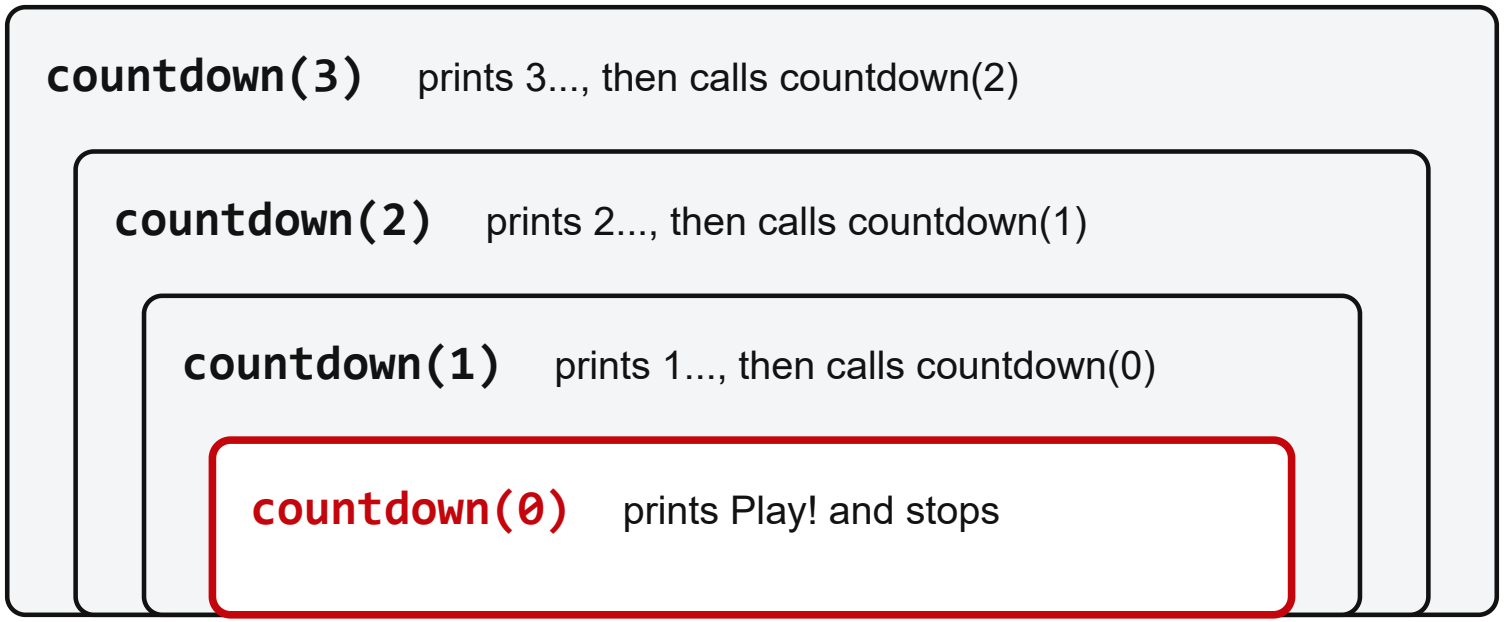


The University of Utah
Programming - Recursion
Recursion is the process of defining a problem (or the

Show all

Recursion

Definition: A **recursive** method calls itself, on a smaller version of the same problem.



Countdown

```
public static void countdown(int n) {
  System.out.println(n + "...");
  countdown(n - 1);               // the same problem, one smaller
}
```

`countdown(3)` should print `3... 2... 1... Play!`

Will it?

Eclipse Exercise

Get today's files from Canvas, and open `Countdown.java`.

Run it. What happens?

```
3...
2...
1...
0...
-1...
   (a few thousand more)
Exception in thread "main" java.lang.StackOverflowError
    at java.base/...                          (Java's own code: skip these)
    at Countdown.countdown(Countdown.java:23)
    at Countdown.countdown(Countdown.java:23)   ...hundreds more
```

StackOverflowError: every call gets a frame on the stack. Each waits for the call it made, so the frames pile up until the stack runs out of room.

Two parts

- **Base case:** a problem small enough to solve right away. No recursive call: it **stops**.
- **Recursive case:** the method calls itself, on a **smaller** problem. Goal is to **converge**.

`countdown` has no base case. What's small enough to solve right away? **`countdown(0)`: just print `Play!`**

Eclipse Exercise

TODO 1. Add a base case to `countdown`. When `n` is 0, print `Play!` and stop with `return;`.

Run it again.

Countdown

```
public static void countdown(int n) {
  if (n == 0) {
    System.out.println("Play!");
    return;                          // base case: stop!
  }
  System.out.println(n + "...");
  countdown(n - 1);                  // recursive case: one smaller
}
```

`3... 2... 1... Play!` - Visualize it!

Tophat Q1

Which call throws a `StackOverflowError`?

```
public static void countdown(int n) {
  if (n == 0) {
    System.out.println("Play!");
    return;
  }
  System.out.println(n + "...");
  countdown(n - 2);
}
```

→ Open Tophat to vote.

Factorial

5! ("5 factorial") multiplies 5 down to 1.

```
public static int calcFactorialIterative(int n) {
  int result = 1;
  for (int i = n; i >= 1; i--) {
    result = result * i;
  }
  return result;
}
```

5 × 4 × 3 × 2 × 1 = **120**. A loop works. Can recursion?

Factorial

```
5! = 5 × 4 × 3 × 2 × 1
   = 5 × (4 × 3 × 2 × 1)
   = 5 × 4!
```

Inside 5! is **4!**: the same problem, one smaller!

What's small enough to solve right away? **1! is just 1.**

Eclipse Exercise

Open `Factorial.java`, and run it. The loop gets 120. Recursion gets 0.

TODO 2. Write `calcFactorialRecursive`. Base case: 1! is 1. Recursive case: n! is n × (n - 1)!

Recursive thinking

How do we find the longest song in a playlist?

- What's the **smallest** problem? One song. It's the longest!
- How do we make the problem **smaller**? Remove a song that can't be the longest.
- How does the smaller answer help? The longest of the rest **is** the longest.

Find the longest song

```
[223, 355, 175, 217]    223 < 355: remove 223
[355, 175, 217]         175 < 355: remove 175
[355, 217]              217 < 355: remove 217
[355]                   one song left: the longest!
```

Is this the best way? **No!** A loop is simpler. But each step is the same problem, **one song smaller**.

Eclipse Exercise

Open `Longest.java`, and run it. `StackOverflowError` again! It prints the same list, over and over.

TODO 3. Get closer to the base case. Before the recursive call, remove the shorter of the first two songs, by its index: 0 or 1.

Longest

```
public static Song longest(ArrayList<Song> songs) {
  if (songs.size() == 1) {
    return songs.get(0);          // base case
  }
  if (songs.get(0).getSeconds() < songs.get(1).getSeconds()) {
    songs.remove(0);              // one song fewer:
  } else {                        //   closer to the base case!
    songs.remove(1);
  }
  return longest(songs);          // recursive case
}
```

Tophat Q2

What does `total` return for the list `[200, 245, 180]`?

```
public static int total(ArrayList<Integer> seconds) {
  if (seconds.size() == 0) {
    return 1;
  }
  int first = seconds.get(0);
  seconds.remove(0);
  return first + total(seconds);
}
```

→ Open Tophat to vote.

Questions?